

POLITECNICO
MILANO 1863

Ingegneria del Software

Ingegneria Informatica – Prof. San Pietro

Esercitazione:

Functional Programming

My Contacts:
Giuseppe Sorrentino
giuseppe.sorrentino@polimi.it

Paradigma Funzionale

- Il paradigma funzionale consiste nel vedere le funzioni da un punto di vista più matematico: $Input \rightarrow Output$
- L'idea è sostituire il linguaggio verbose delle classi anonime, molto usate prima di java 8, con espressioni Lambda e funzioni.

Lambda Expression: Some Rules

- L’input di una lambda expression DEVE essere immutabile.

Lambda Expression: Some Rules

- L’input di una lambda expression DEVE essere immutabile.
- Il tipo dell’input può essere inferito

Lambda Expression: Some Rules

- L’input di una lambda expression DEVE essere immutabile.
- Il tipo dell’input può essere inferito
- Se l’espressione Lambda ha un solo statement, il return può essere implicito
  object.method((par1, par2) -> par1.foo(par2))

Lambda Expression: Some Rules

- L'input di una lambda expression DEVE essere immutabile.
- Il tipo dell'input può essere inferito
- Se l'espressione Lambda ha un solo statement, il return può essere implicito
  object.method((par1, par2) -> par1.foo(par2))
- Se l'espressione Lambda ha più di uno statement, il return deve essere esplicito

```
object.method(final String name –> {
        if (name.equals(“Bob”)) return “Foo”;
        else return name;
});
```

Optional <T> Type

- Un oggetto di tipo Optional<T> può essere usato senza dover controllare che non sia null. Infatti, se l'oggetto è Optional esistono I seguenti metodi:

  - **ifPresent(T t -> doSomething)** prende in input una funzione che accetta T e lo usa

  - **flatMap(function)** prende in inup una funzione cha ha T come input e restituisce un Optional

  - orElse(T val) se l'oggetto che la chiama NON è empty, restituisce il suo valore, altrimenti restituisce "val"

Stream <T>

- Permettono di concatenare funzioni che agiscono sulle collections
- Contengono molti metodi utili che possono essere utilizzati per «lavorare» con le collections.

Stream <T> : forEach()

- Il metodo **forEach()** permette di «far qualcosa» con ogni elemento dello stream.

  Es:
  List <String> list = …. // inizializzazione della lista di stringhe

  opzione1: list.stream().forEach(System.out::println)
  opzione2: list.stream().forEach(x -> System.out.println(x))

- Cosa cambia?

Stream <T> : forEach()

- La programmazione funzionale viene solitamente usata con le Lambda Expression che sono delle funzioni anonime (opzione 1). Tuttavia, posso usarla anche con funzioni già esistenti (opzione 2).

Es:
List <String> list = …. // inizializzazione della lista di stringhe

opzione1: list.stream().forEach(x -> System.out.println(x))
opzione2: list.stream().forEach(System.out::println)

Stream <T> : filter()

- Il metodo **filter()** prende in input un predicato, ovvero un'espressione che restituisce un booleano, e restituisce uno stream contenente SOLO gli elementi dello stream di input che rispettano il predicato dato.

Es:

```
List <Integer> list = .... // inizializzazione della lista di stringhe

list.stream().filter(i -> i % 2 == 1)
```

Stream <T> : map()

- Il metodo **map()** per ogni elemento di tipo T dello stream su cui viene applicato, fa un'operazione e restituisce un elemento di tipo U.
  U e T possono essere sia uguali, che diversi.

Es:

```
List <String> list = .... // inizializzazione della lista di stringhe

list.stream().map(i -> i.length())
```

Stream <T> : distinct() e sorted()

- Il metodo **distinct()** elimina i duplicati dallo Stream<T> su cui è applicato
- Il metodo **sorted()** riordina gli elementi dello Stream<T> su cui è applicato. Se usato su uno stream di interi o stringhe, restituisce un ordine «normale», mentre richiede un Comparator per qualsiasi altro tipo.

  Es: restituire le lunghezze delle stringhe ordinate e senza ripetizioni
  List <String> list = …. // inizializzazione della lista di stringhe

  list.stream().map(x –> x.length).distinct().sorted()
- Come risulta evidente da questo esempio, **l'ordine con cui applichiamo le funzioni è rilevante.**

Stream <T> : flatMap()

- Potete immaginarla come una **map** che lavora su tipi di dato più complessi.
- Per ogni elemento T dello stream di input, **flatMap()** fa un'operazione e restituisce uno stream<U> **per ogni elemento.**
- Tutti questi stream<U> vengono poi appiattiti e concatenati, per ottenere uno stream unico di elementi di tipo U

- Es:

```
List<String> frasi = Arrays.asList("Ciao mondo", "Programmazione funzionale", "Lambda expressions");
// Utilizziamo flatMap per ottenere un singolo stream di parole da queste frasi
 List<String> parole = frasi.stream()
                        .map(frase -> frase.split(" ")) // Divide le frasi in array di parole
                        .flatMap(Arrays::stream) // Appiattisce gli array di parole in uno stream singolo
```

Stream <T> : reduce()

- Consente di ottenere, dato uno stream<T>, un solo valore
- La funzione che viene usata per restituire questo valore può essere già esistente nel tipo di stream che si sta usando (ad esempio gli intStream() hanno average e sum), oppure può essere data come input.
- Se la funzione da usare è un input, reduce prende come primo parametro il valore iniziale e come secondo parametro la funzione

- Es:

```
List<Integer> list = ….
Integer sum  = list.stream().reduce(0.0 , (a,b) -> a+b);
```

Stream <T> : collect()

- **Collect()** prende in input 3 funzioni.
  - Supplier: essenzialmente è la funzione che «crea» il valore di ritorno
  - Accumulator: è la funzione che aggiunge qualcosa al valore di ritorno
  - Combine: è la funzione che «combina» due valori di ritorno
- Per le collection «standard» esistono metodi pre-implementati, ma possiamo comunque implementarne noi uno.

Es 1:

```
List <String> sentence = …
List <String> wordsWithA = sentence.stream().filter(w -> w.startsWith(«A»))
                                            . collect(Collectors.toList())
```

Es 2:

```
List <String> wordsWithA = sentence.stream().filter(w -> w.startsWith(«A»))
                                            . collect(ArrayList::new, ArrayList::add,
                                            ArrayList :: addAll)
```

Note:

- Le operazioni degli stream sono Stateless, **non** possono modificare lo stream su cui sono applicate.

Note:

- Le operazioni degli stream sono Stateless, **non** possono modificare lo stream su cui sono applicate.
- Le operazioni degli stream possono restituire **un nuovo stream** con il risultato.

Note:

- Le operazioni degli stream sono Stateless, **non** possono modificare lo stream su cui sono applicate.
- Le operazioni degli stream possono restituire **un nuovo stream** con il risultato.
- Le operazioni degli stream sono **lazy**, ovvero vengono eseguite solo quando serve.

Note:

- Le operazioni degli stream sono Stateless, **non** possono modificare lo stream su cui sono applicate.
- Le operazioni degli stream possono restituire **un nuovo stream** con il risultato.
- Le operazioni degli stream sono **lazy**, ovvero vengono eseguite solo quando serve.
- Se vogliamo usare una Lambda Function più volte, possiamo assegnarla ad una variabile di tipo Predicate <T>
  Es: final Predicate <String> funcStart = s -> s.startsWith(«A»);

Note:

- Possiamo dichiarare un metodo che restituisce un Predicate <T>
  Es:

```
public static Predicate<T> starter (String prefisso){
        return s -> s.startWith(prefisso);

list.stream().filter(pred(«G»));
```

Note:

- Possiamo dichiarare un metodo che restituisce un Predicate <T>
  Es:

```
public static Predicate<T> starter (String prefisso){
        return s -> s.startWith(prefisso);

list.stream().filter(pred(«G»));
```

- Una Lambda Expression che usa una variabile non definita all'interno del suo scope, è chiamata Closure.

Note:

- Possiamo dichiarare un metodo che restituisce un Predicate <T>
  Es:

```
public static Predicate<T> starter (String prefisso){
        return s -> s.startWith(prefisso);

list.stream().filter(pred(«G»));
```

- Una Lambda Expression che usa una variabile non definita all'interno del suo scope, è chiamata Closure.
- Per evitare effetti indesiderati, una variabile usata da una Lambda Expression dovrebbe essere **final** o comunque non cambiare valore.

Esercizi: TdE 16/01/18

Esercizio 3

Si consideri la classe `Couple` così definita:

```
public class Couple {
    public int val1;
    public int val2;
}
```

e una lista `data` di istanze di tale classe, opportunamente inizializzata:

```
List<Couple> data = new ArrayList<Couple>();
data.add(...); ...; data.add(...);
```

Si scriva un frammento di programma Java 8 che usa lo stile funzionale per restituire la somma di tutti i secondi elementi (`val2`) delle coppie nella lista `data` il cui primo elemento sia maggiore o uguale a zero.

Esercizi: TdE 16/01/18

- Soluzione:

```
int result = data.stream().filter( c -> c.val1 >= 0)
                          .reduce(0.0, (tot, c) -> tot+c.val2)
```

- Oppure:

```
int result = data.stream().filter(c -> c.val1 >= 0)
                          .map( c -> c.val2)
                          .reduce(0.0, (tot,c) -> tot = tot+c)
```

Esercizi: TdE 02/09/16

Trasformare il seguente codice Java in stile funzionale.

```
public static void numeroPari(final List<Integer> numeri) {

  Integer found = null;
  for (Integer i: numeri) {
    if (i % 2 ==0) {
      found = i;
      break;

    }
  }
  if (found != null) {
    System.out.print("Il primo numero pari " + found);
  } else {
    System.out.println("Nessun numero pari!");
  }
}
```

Esercizi: TdE 02/09/16

Trasformare il seguente codice Java in stile funzionale.

```
public static void numeroPari(final List<Integer> numeri) {

  Integer found = null;
  for (Integer i: numeri) {
    if (i % 2 ==0) {
      found = i;
      break;
    }
  }
  if (found != null) {
    System.out.print("Il primo numero pari " + found);
  } else {
    System.out.println("Nessun numero pari!");
  }
}
```

Hint: abbiamo un controllo sul valore **null**

Esercizi: TdE 02/09/16

Soluzione

```
Optional <Integer> trovato = numeri.stream().filter(i->i%2==0).findFirst()

If (trovato.ifPresent()){
        System.out.println('il primo numero pari è:' + found);
Else
        System.out.println('non è stato trovato alcun numero pari');
```

Esercizi: TdE 16/09/16

Esercizio 3

Si consideri la seguente definizione di una classe Person:

```
public class Person {
  private final String name;
  private final int age;
  public Person(final String theName, final int theAge) {
    name = theName;
    age = theAge;
  }
  public int ageDifference(final Person other) {
    return age - other.age;
 }
  public String toString() {
    return String.format("%s - %d", name, age);
  }
...
}
```

Il seguente frammento di codice definisce un ArrayList people di Person e, a mero titolo di esempio, lo inizializza con alcuni valori.

```
final List<Person> people = Arrays.asList(
  new Person("John", 20),
  new Person("Sara", 21),
  new Person("Jane", 21),
  new Person("Greg", 35));
```

Si rammenta qui la definizione in Java 8 del notissimo interfaccia Comnparator:

```
@FunctionalInterface
public interface Comparator<T> {
  // \result<0 se o1 minore di o2, =0 se uguali, altrimenti >0
  int compare(T o1, T o2)
  ...
}
```

Scrivere un frammento di programma Java 8 in stile funzionale che stampa le persone contenute in people in ordine di eta' decrescente (nell'esempio, Greg, Sara, Jane, John).

Si suggerisce di utilizzare l'operazione `sorted()` definita per tutti gli stream, che prevede un argomento funzionale che sia un Comparator. Ad esempio, se la variabile stringCollection è una List di oggetti String (la cui classe fornisce un metodo compareTo), il seguente frammento definisce uno stream ordinato in ordine crescente.

```
stringCollection
   .stream()
   .sorted((a, b) -> b.compareTo(a))
```

- Hint1: chiede di stampare, quindi si usa qualcosa di già noto alla fine del functional.
- Hint2: chiede un ordine, quindi servirà un Comparator in base all'oggetto dato. Controllare se esistono metodi della classe che possono aiutarvi.
- Hint3: il comparatore è predefinito **solo** per interi e stringhe.

Esercizi: TdE 16/09/16

Soluzione

```
people.stream().sorted( (p1,p2) -> {
if ( p1.ageDifference(p2) > 0) return 1;
else{
   if (p1.ageDifference(p2) < 0) return -1;
   else
         return 0;
}
}).forEach(System.out::print);
```

Esistono soluzioni più compatte?

Esercizi: TdE 16/09/16

Soluzione

```
people.stream()
        .sorted((p1,p2) -> p1.ageDifference(p2)).forEach(System.out::println);
```

Esercizi: TdE 16/09/16

Soluzione

```
people.stream()
        .sorted((p1,p2) -> p1.ageDifference(p2)).forEach(System.out::println);
```

oppure

```
people.stream().sorted(Person::ageDifference).forEach(System.out.println);
```

Esercizi: TdE 16/09/16

Soluzione

```
people.stream()
        .sorted((p1,p2) -> p1.ageDifference(p2)).forEach(System.out::println);
```

oppure

```
people.stream().sorted(Person::ageDifference).forEach(System.out.println);
```

Infatti, di fatto stiamo usando una funzione ben definita, quindi non è necessario utilizzare una lambda, che di fatto implementa una funzione anonima che rispetta l'interfaccia Comparator

Esercizi: TdE 21/07/22

Esercizio 4

Si considerio le seguenti classi Java, in cui viene omessa l'ovvia implementazione dei metodi osservatori `getMembers()`, `getAge()`, `getName()`.

```
public class Group {
  public List<Person> getMembers() { ... }
}

class Person {
  public String getName() { ... }
  public int getAge() { ... }
}
```

Si risponda ai seguenti quesiti facendo uso *esclusivamente* di costrutti della programmazione funzionale.

Domanda a)

Si completi l'implementazione del seguente metodo per fare in modo che stampi l'età media delle persone appartenenti al gruppo. Se il gruppo è vuoto, il metodo non deve stampare nulla.

```
public static void printAverage(Group group) {
  ...
}
```

Esercizi: TdE 21/07/22

Soluzione:

```
public static void printAverage(Group group){
        group.getMembers().stream()
                                .mapToInt(p -> p.getAge())
                                 .average()
                                 .ifPresent(System.out::println)
}
```

dato che chiede di usare solo costrutti funzionali, occorre ricordare che average restituisce un'opzionale e che questo può essere controllato senza usare if statement. Inoltre, occorre ricordare che per usare average() dobbiamo lavorare con un intStream, quindi usare mapToInt()

Esercizi: TdE 21/07/22

Domanda b)

Si completi l'implementazione del seguente metodo per fare in modo che ritorni una lista contenente i nomi di tutte le persone maggiorenni presenti all'interno dei gruppi in `groups`.

```
public static List<String> namesInGroups(List<Group> groups) {
  return ...
}
```

Esercizi: TdE 21/07/22

Soluzione:
1. Per ogni gruppo, mi serve uno stream di Persone. Tutti questi stream vanno poi messi insieme in un'unica lista. Quindi mi serve una flatMap

```
groups.stream()
        .flatMap( g -> g.getMembers().stream())
```

Esercizi: TdE 21/07/22

Soluzione:
2. Mi interessano soltanto persone maggiorenni, quindi devo filtrare ogni stream per età

```
groups.stream()
        .flatMap( g -> g.getMembers().stream())
        .filter(p -> p.getAge() >= 18)
```

Esercizi: TdE 21/07/22

Soluzione:
3. Ora ho uno stream che contiene solo persone maggiorenni. Ma io non voglio persone, soltanto nomi. Quindi faccio un map

```
groups.stream()
        .flatMap( g -> g.getMembers().stream())
        .filter(p -> p.getAge() >= 18)
        .map(p -> p.getNome())
```

Esercizi: TdE 21/07/22

Soluzione:

4. A questo punto, per ogni gruppo della lista di gruppi ho ottenuto uno stream di nomi di persone maggiorenni. Ma io voglio una lista!

```
groups.stream()
        .flatMap( g -> g.getMembers().stream())
        .filter(p -> p.getAge() >= 18)
        .map(p -> p.getNome())
        .collect(Collectors.toList());
```

Esercizio Extra: Registro di classe

Un registro di classe contiene:

- La lista dei voti (decimali da 0 a 10) …
- … in ogni materia (enum Subject) ...
- … per ogni studente (stringa)

```
Map<String, Map<Subject, List<Double>>> records
```

Esercizio Extra: Registro di classe

Secondo il paradigma funzionale, estrarre lo studente con la media più alta per ciascuna materia.

```
Map<Subject, String>
```

Esercizio Extra: Registro di classe

Si supponga che il registro sia popolato nel seguente modo:

```
Map<Subject, List<Double>> alanMarks = new HashMap<>();
alanMarks.put(Subject.MATH, Arrays.asList(9.5, 8.0, 9.0, 9.25));
alanMarks.put(Subject.SCIENCE, Arrays.asList(7.25, 9.50, 5.75));

Map<Subject, List<Double>> brianMarks = new HashMap<>();
brianMarks.put(Subject.MATH, Arrays.asList(6.25, 9.0, 8.75));
brianMarks.put(Subject.SCIENCE, Arrays.asList(10.0));

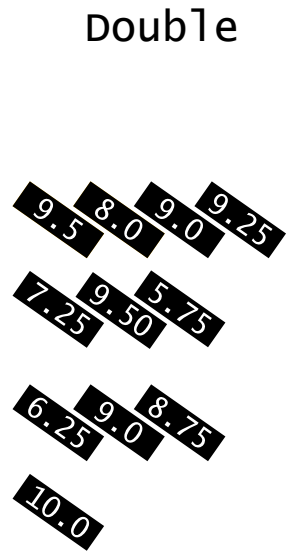
Map<String, Map<Subject, List<Double>>> records = new HashMap<>();
records.put("Alan", alanMarks);
records.put("Brian", brianMarks);
```

Esercizio Extra: Registro di classe

```
Map<Subject, List<Double>> alanMarks = new HashMap<>();
alanMarks.put(Subject.MATH, Arrays.asList(9.5, 8.0, 9.0, 9.25));
alanMarks.put(Subject.SCIENCE, Arrays.asList(7.25, 9.50, 5.75));

Map<Subject, List<Double>> brianMarks = new HashMap<>();
brianMarks.put(Subject.MATH, Arrays.asList(6.25, 9.0, 8.75));
brianMarks.put(Subject.SCIENCE, Arrays.asList(10.0));

Map<String, Map<Subject, List<Double>>> records = new HashMap<>();
records.put("Alan", alanMarks);
records.put("Brian", brianMarks);
```



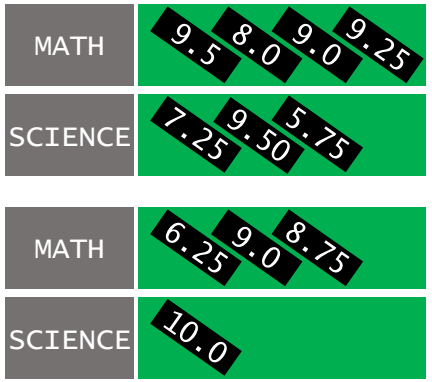
Esercizio Extra: Registro di classe

```
Map<Subject, List<Double>> alanMarks = new HashMap<>();
alanMarks.put(Subject.MATH, Arrays.asList(9.5, 8.0, 9.0, 9.25));
alanMarks.put(Subject.SCIENCE, Arrays.asList(7.25, 9.50, 5.75));

Map<Subject, List<Double>> brianMarks = new HashMap<>();
brianMarks.put(Subject.MATH, Arrays.asList(6.25, 9.0, 8.75));
brianMarks.put(Subject.SCIENCE, Arrays.asList(10.0));

Map<String, Map<Subject, List<Double>>> records = new HashMap<>();
records.put("Alan", alanMarks);
records.put("Brian", brianMarks);
```

```
Subject,
List<Double>
```



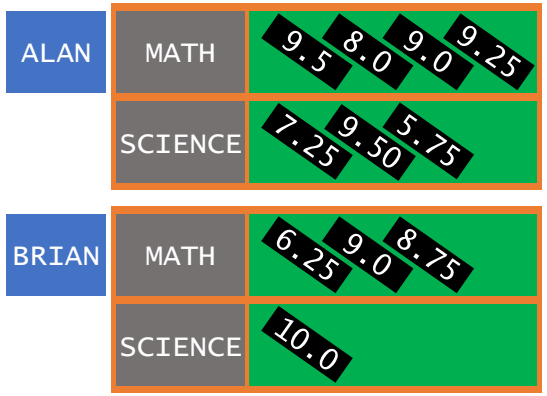
Esercizio Extra: Registro di classe

```
Map<Subject, List<Double>> alanMarks = new HashMap<>();
alanMarks.put(Subject.MATH, Arrays.asList(9.5, 8.0, 9.0, 9.25));
alanMarks.put(Subject.SCIENCE, Arrays.asList(7.25, 9.50, 5.75));

Map<Subject, List<Double>> brianMarks = new HashMap<>();
brianMarks.put(Subject.MATH, Arrays.asList(6.25, 9.0, 8.75));
brianMarks.put(Subject.SCIENCE, Arrays.asList(10.0));

Map<String, Map<Subject, List<Double>>> records = new HashMap<>();
records.put("Alan", alanMarks);
records.put("Brian", brianMarks);
```

```
Map<
   Subject,
   List<Double>
>
```



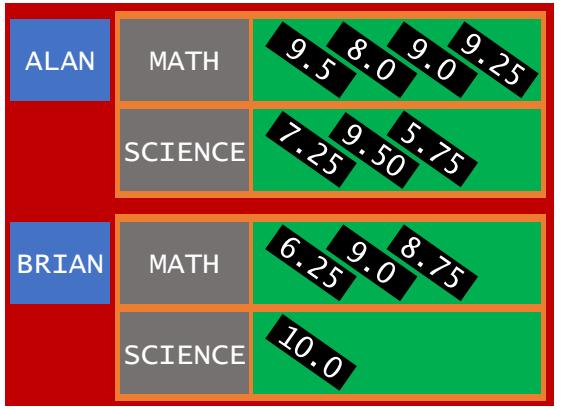
Esercizio Extra: Registro di classe

```
Map<
  String,
    Map<
      Subject,
      List<Double>
    >
>
```

```
Map<Subject, List<Double>> alanMarks = new HashMap<>();
alanMarks.put(Subject.MATH, Arrays.asList(9.5, 8.0, 9.0, 9.25));
alanMarks.put(Subject.SCIENCE, Arrays.asList(7.25, 9.50, 5.75));

Map<Subject, List<Double>> brianMarks = new HashMap<>();
brianMarks.put(Subject.MATH, Arrays.asList(6.25, 9.0, 8.75));
brianMarks.put(Subject.SCIENCE, Arrays.asList(10.0));

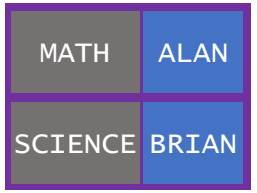
Map<String, Map<Subject, List<Double>>> records = new HashMap<>();
records.put("Alan", alanMarks);
records.put("Brian", brianMarks);
```



Esercizio Extra: Registro di classe

Secondo il paradigma funzionale, estrarre lo studente con la media più alta per ciascuna materia.

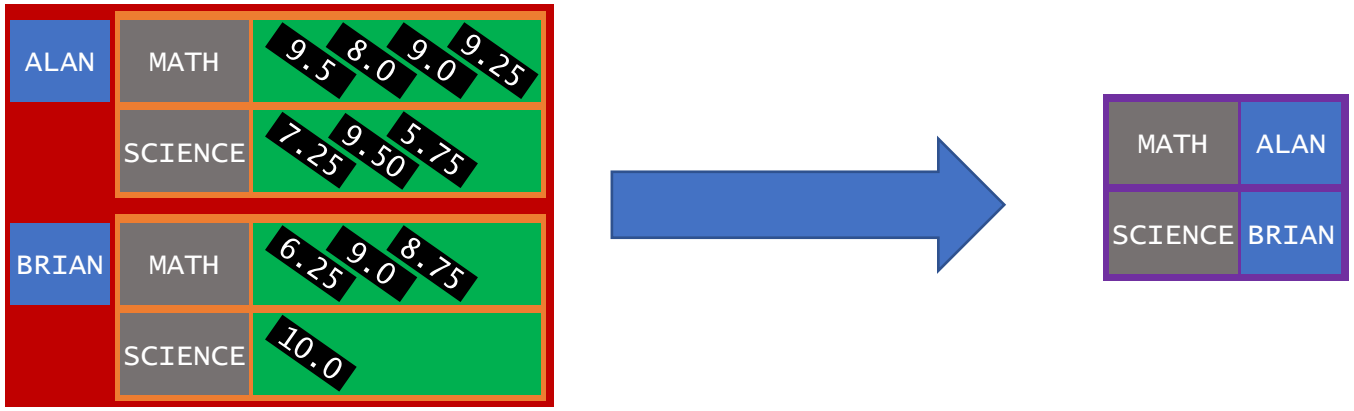
```
Map<Subject, String>
```



Esercizio Extra: Registro di classe

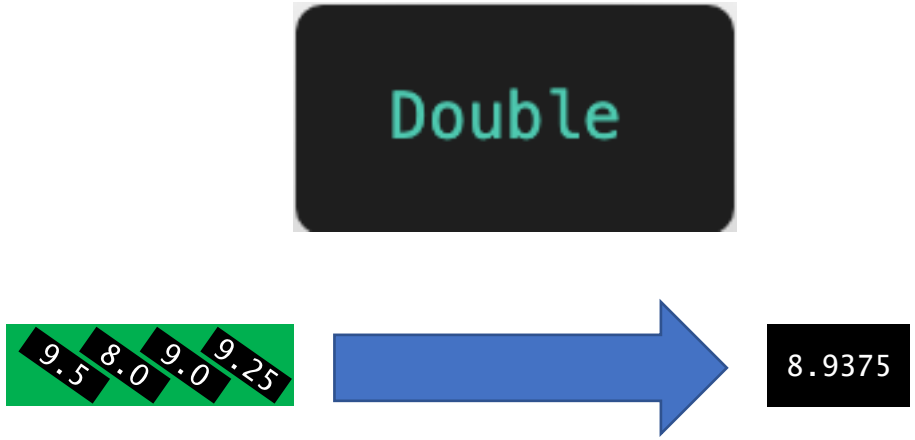
```
Map<
  String,
    Map<
      Subject,
      List<Double>
    >
>
```

```
Map<
 Subject,
 String
>
```



Esercizio Extra: Registro di classe (STEP 1)

Secondo il paradigma funzionale, estrarre la media dei voti di «Alan» in «Subject.MATH»



Esercizio Extra: Registro di classe (STEP 1)

Secondo il paradigma funzionale, estrarre la media dei voti di «Alan» in «Subject.MATH»

```
Double avg =
            records.get("Alan").get(Subject.MATH)
                .stream()
                .reduce(0.0, (a, b) -> a + b)

                /

            records.get("Alan").get(Subject.MATH).size();
```

… prima di continuare …

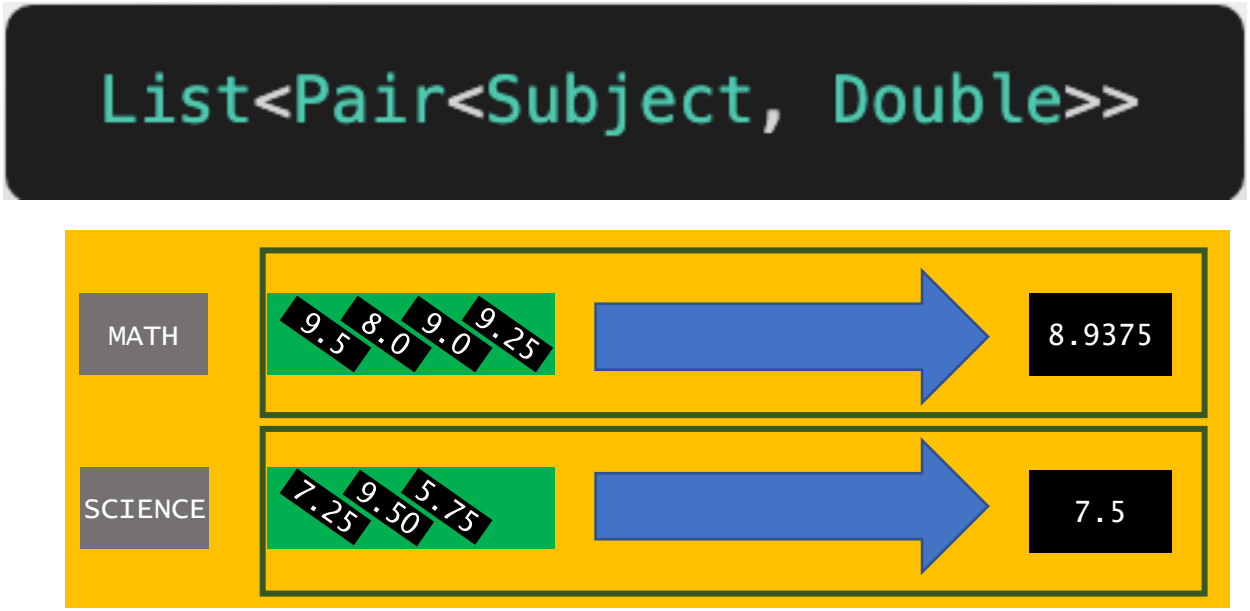
```java
public class Pair<A, B> {
    public final A _1;
    public final B _2;

    public Pair(A _1, B _2) {
        this._1 = _1;
        this._2 = _2;
    }

    @Override
    public String toString() {
        return "(" + (_1 != null ? _1.toString() : "null") + ", "
                   + (_2 != null ? _2.toString() : "null") + ")";
    }
}
```

Esercizio Extra: Registro di classe (STEP 2)

Secondo il paradigma funzionale, estrarre la lista delle coppie (Materia, Media) per «Alan»



Esercizio Extra: Registro di classe (STEP 2)

```
List<Pair<Subject, Double>> avgAndSubject =
            records.get("Alan")
                .entrySet()
                .stream()
                .map(
                    (entry) -> {
                        return new Pair<Subject, Double>(
                            entry.getKey(),
                            (
                                entry.getValue()
                                    .stream()
                                    .reduce(0.0, (a, b) -> a + b)
                                        /
                                entry.getValue().size()
                            )
                        );
                    }
                ).collect(Collectors.toList());
```

… prima di continuare …

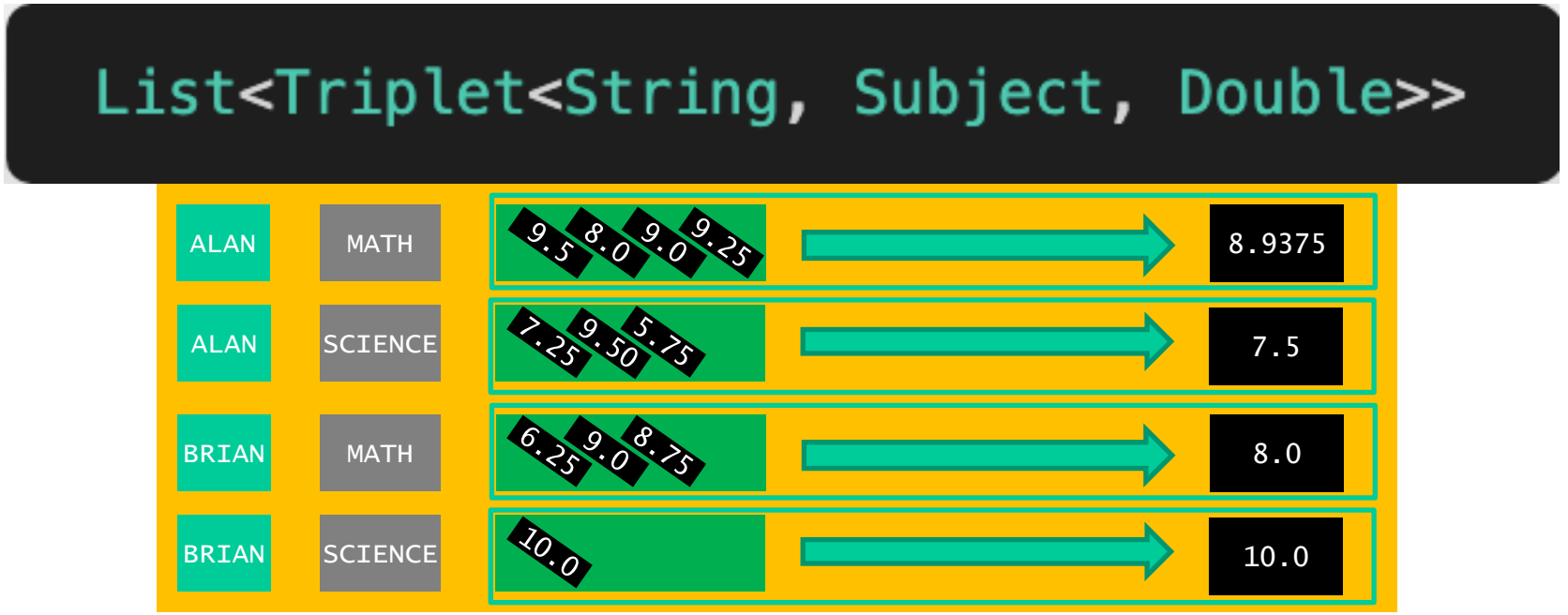
```java
public class Triplet<A, B, C> {
    public final A _1;
    public final B _2;
    public final C _3;

    public Triplet(A _1, B _2, C _3) {
        this._1 = _1;
        this._2 = _2;
        this._3 = _3;
    }

    @Override
    public String toString() {
        return "(" + ( _1 != null ? _1.toString() : "null" ) + ", "
                   + ( _2 != null ? _2.toString() : "null" ) + ", "
                   + ( _3 != null ? _3.toString() : "null" ) + ")";
    }
}
```

Esercizio Extra: Registro di classe (STEP 3)

Secondo il paradigma funzionale, estrarre la lista delle triplette (Studente, Materia, Media)



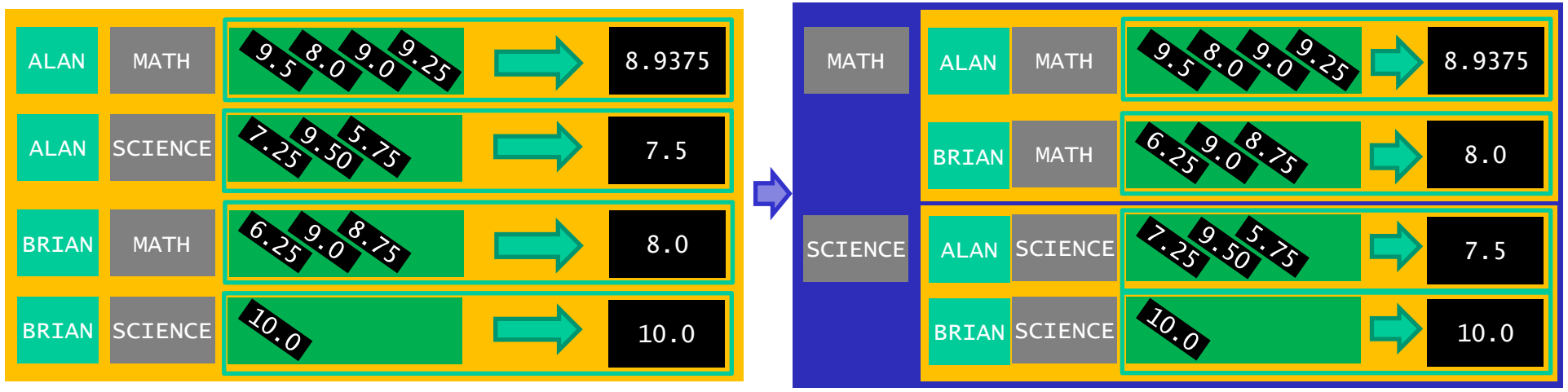
Esercizio Extra: Registro di classe (STEP 3)

```
List<Triplet<String, Subject, Double>> avgAndPersonAndSubject =
            records
                .entrySet()
                .stream()
                .flatMap(
                    (record) -> {
                        return record
                        .getValue()
                        .entrySet()
                        .stream()
                        .map(
                            (entry) -> {
                                return new Triplet<String, Subject, Double>(
                                    record.getKey(),
                                    entry.getKey(),
                                    (
                                        entry.getValue()
                                            .stream()
                                            .reduce(0.0, (a, b) -> a + b)
                                                /
                                        entry.getValue().size()
                                    )
                                );
                            }
                        );
                    }
                ).collect(Collectors.toList());
```

Esercizio Extra: Registro di classe (STEP 3)

Secondo il paradigma funzionale, estrarre la lista delle triplette (Studente, Materia, Media) raggruppate per Materia

```
Map<Subject, List<Triplet<String, Subject, Double>>>
```



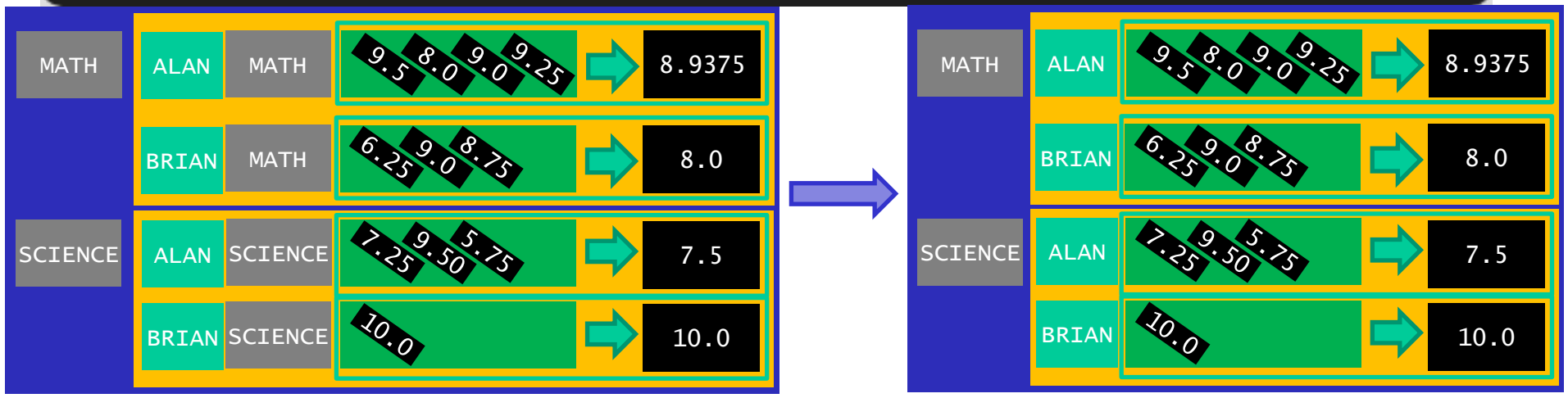
Registro di classe – STEP 4

```
Map<Subject, List<Triplet<String, Subject, Double>>>
avgAndPersonAndSubjectPerSubject =
    records
        .entrySet()
        .stream()
        .flatMap(
            (record) -> {
                return record
                .getValue()
                .entrySet()
                .stream()
                .map(
                    (entry) -> {
                        return new Triplet<String, Subject, Double>(
                            record.getKey(),
                            entry.getKey(),
                            (
                                entry.getValue()
                                    .stream()
                                    .reduce(0.0, (a, b) -> a + b)
                                        /
                                entry.getValue().size()
                            )
                        );
                    }
                );
            }
        ).collect(Collectors.groupingBy(
            (record) ->
                record._2
        ));
```

Esercizio Extra: Registro di classe (STEP 5)

Secondo il paradigma funzionale, estrarre la lista delle coppie (Studente, Media) raggruppate per Materia

```
Map<Subject, List<Pair<String, Double>>>
```



Registro di classe – STEP 5



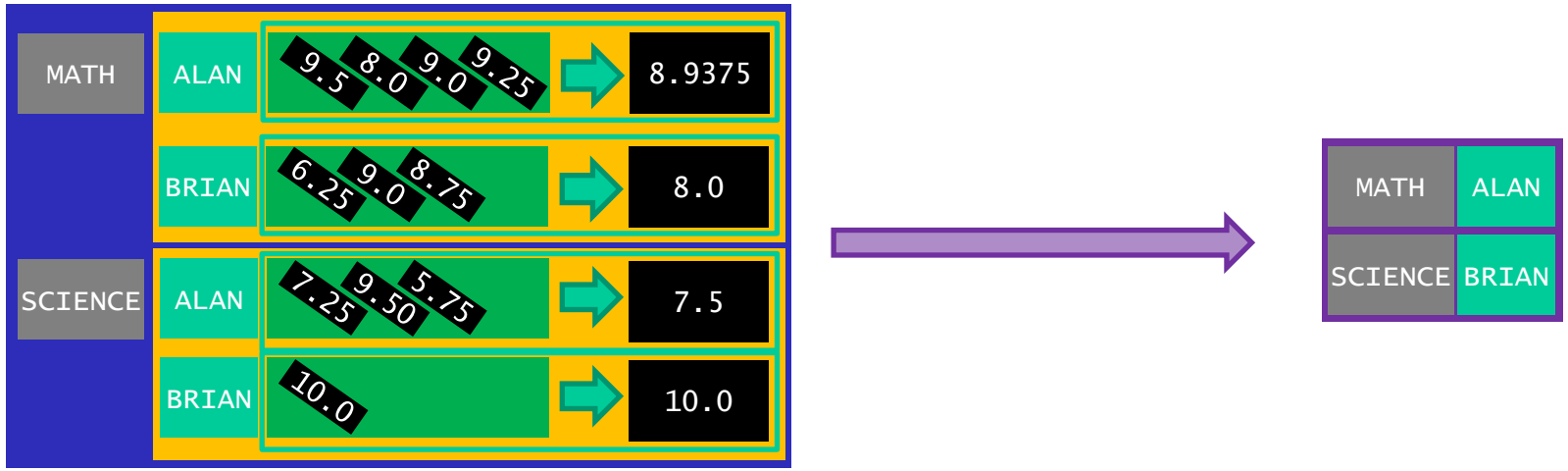
Continuo dallo STEP 4…

```
.entrySet()
                .stream()
                .map(
                    (record) -> {
                        List<Pair<String, Double>> val =
                            record.getValue()
                            .stream()
                            .map(
                                (triplet) ->
                                    new Pair<>(triplet._1, triplet._3)
                            ).collect(Collectors.toList());
                        return new Pair<>(record.getKey(), val);
                    }
                ).collect(Collectors.toMap(
                    record -> record._1,
                    record -> record._2)
                );
```

Esercizio Extra: Registro di classe (Conclusione)

Secondo il paradigma funzionale, estrarre lo studente con la media più alta per ciascuna materia.

```
Map<Subject, String>
```



Registro di classe – CONCLUSIONE



Continuo dallo STEP 5…

```
.entrySet()
                .stream()
                .map(
                    (subject) -> {
                        return new Pair<>(
                            subject.getKey(),
                            subject.getValue()
                                .stream()
                                .reduce(new Pair<>(null, 0.0),
                                    (acc, val) -> {
                                        if (val._2 > acc._2) {
                                            return val;
                                        } else {
                                            return acc;
                                        }
                                    }
                                )
                                ._1
                        );
                    }
                ).collect(Collectors.toMap(
                    subject -> subject._1,
                    subject -> subject._2)
                );
```

Ack

- Esercizi di Andrea Damiani, estesi da Giuseppe Sorrentino
- Teoria: slide prof. San Pietro